WHITE PAPER · 23 OCTOBER 2026

Owning Your AI: Hardware and Training for Private Local Models

Christopher Maglio and Muse

A practical guide for the novice developer: what it actually takes — in memory, in hardware, in training strategy — to run a private AI you own, on machines you control. Written for readers who want the full chain, from the mathematics to the purchase decision, in one handy-sized document.

Abstract

Running an AI model privately — on your own hardware, with your own data, answerable to no cloud provider — is one of the most attractive ideas in modern computing and one of the most misunderstood. This paper explains, in plain terms, what you are getting into. First, *why* AI demands so much memory: inference is a memory-bandwidth problem, not a compute problem, because every token generated requires streaming the model's full weight matrices through the processor. Second, *what* that means for hardware: a concrete map from model size and numerical precision to RAM requirements, worked through two real models — Meta's 30-billion-parameter Muse Glimmer, which fits on a deskside machine, and the 816-billion-parameter Muse Spark Open, which does not fit on anything you can reasonably buy. Third, *how* to adapt a model once it runs: a size-dependent strategy for retrieval-augmented generation, low-rank adaptation, and full training, and why the right choice is driven above all by the size of the base model. The paper closes with the economic consequence nobody planned: the insatiable demand for memory that local AI has helped create.

1. Why AI Needs So Much Memory

Every newcomer to local AI asks some version of the same question: why does this need hundreds of gigabytes of RAM when my other programs need megabytes? The answer is in the mathematics, and it is worth understanding before spending any money, because it explains every hardware decision that follows.

A large language model is, at its core, a collection of very large two-dimensional arrays of numbers — matrices, called *tensors* — plus a few one-dimensional arrays of biases and normalization scales. A 30-billion-parameter model is roughly 30 billion numbers arranged mostly as big matrices. To generate *one token* (about three-quarters of an English word), the processor must multiply the current state by *every one of those matrices*. Every number in the model is touched, every token, in sequence. There is no shortcut: the computation for token N+1 depends on the output of token N, so the work cannot be parallelized across tokens the way other workloads parallelize across data.

This has a decisive consequence: **inference is limited by memory bandwidth, not by computing power.** The processor spends most of its time waiting for numbers to arrive from memory, not multiplying them. Doubling your processor's speed barely helps; doubling the speed at which memory delivers weights doubles your token rate. A useful rule of thumb: divide the memory bandwidth (in gigabytes per second) by the size of the weights that must stream per token (in gigabytes) and you get a ceiling on tokens per second. Everything else — the choice of GPU, the unified-memory architecture, the reason swap files are useless — follows from this one fact.

Swap deserves a special warning because it is the most common novice mistake. An operating system can pretend to have more RAM by paging data to a drive, and for ordinary programs this works invisibly. For AI inference it does not, because the drive is hundreds of times slower than RAM and *every token needs the whole model*. A model that generates 30 tokens per second from RAM generates one token per minute from a fast SSD — and one per several minutes from a slow one. Swap solves capacity. It does not solve bandwidth. There is no configuration of drives, buses, or cleverness that changes this; the pipes are three orders of magnitude narrower than what the mathematics requires.

2. Why This Is Nothing Like Bitcoin Mining

The comparison comes up constantly, because Bitcoin mining is the one precedent most people know for "ordinary people running serious computation at home." The two workloads could hardly be more different, and the difference explains why AI will never distribute the way mining did.

Bitcoin hashing is *embarrassingly parallel*: each attempt at a hash is independent of every other attempt, the state involved is tiny (a block header of 80 bytes), and ten thousand weak machines each trying different nonces contribute exactly as much as one strong machine. Any sliver of compute earns a proportional sliver of reward. That is why mining spread to garages and botnets.

AI inference is the opposite in every respect. The state is the entire model — tens to hundreds of gigabytes — and it cannot be subdivided across a network usefully, because every token needs all of it *in one place, at memory speed*. You cannot pool the RAM of a thousand home computers over the internet; the interconnect is billions of times too slow. A phone or a small board contributing a fraction of a model's weights is not a fraction of the work — it is nothing, because the token cannot be generated until the whole matrix has streamed past the processor. AI compute comes in indivisible, memory-sized chunks. That is why there is no AI equivalent of the mining pool for weak hardware, and why the economics of local AI are the economics of *memory*, not of processors.

3. The Precision Ladder: What Quantization Buys You

The numbers in a model are stored with some *precision* — some number of bits per number — and precision is the main dial you turn to fit a model into available memory. The standard

ladder runs from 16-bit floating point (the precision models are trained and released at), through 8-bit, down to 4-bit, with 2-bit at the ragged edge. Each step roughly halves the memory footprint. Each step also costs some intelligence: fewer bits per weight means more rounding error, and while neural networks are remarkably tolerant of rounding, the tolerance is not infinite.

For orientation, 4-bit is the sweet spot the industry has settled on: large memory savings for a small quality cost. Below 4-bit, quality falls off sharply — 2-bit cuts into signal, not just noise. Mixed schemes exist that keep sensitive weights at higher precision, and per-block scaling formats (such as NVIDIA's NVFP4) place each small group's limited precision window over the part of the value range that matters locally, narrowing the gap between 4-bit and 8-bit without closing it.

Meta's published figures for Muse Glimmer illustrate the trade concretely (as reported by Meta; we have not independently verified them): the **4-bit quantized model degrades about 1%** relative to full precision, and the **8-bit model about 0.2%**. One percent is small enough to ignore for most purposes and large enough to notice on hard reasoning — which is exactly why the ladder matters: you choose your precision by deciding how much quality you are willing to spend for memory you do not have.

4. The Hardware Map: Glimmer as a Worked Example

Muse Glimmer is a 30-billion-parameter dense model released under the Apache 2.0 license — free to run, modify, and train. It is the natural reference point for private AI: capable enough to be genuinely useful, small enough to fit on hardware a person can own. Here is what it demands at each precision, in round numbers:

Precision	Weights on disk/RAM	What runs it
4-bit	~17–20 GB	A 24 GB GPU (e.g. RTX 3090/4090); high-memory mini-PCs
8-bit	~30–34 GB	48 GB GPUs; 64 GB unified-memory machines
16-bit (full)	~55–60 GB	128 GB unified-memory deskside (e.g. NVIDIA DGX Spark)

Table 1 — Approximate memory footprint of Muse Glimmer by precision, with representative hardware. Add headroom for context (long conversations consume gigabytes) and the operating system.

Two observations for the buyer. First, **buy memory, not compute**: within a generation, the card or machine with more RAM beats the faster one with less, because bandwidth over the weights is the binding constraint. Second, full 16-bit precision buys measurably better decisions — it is the best the model can be — but the gain over 4-bit is a few benchmark points, visible on hard reasoning and long agentic chains, invisible in everyday use. Most private deployments run 4-bit and never look back.

> **A note on training memory.** Running a model is one thing; *training* it is another, and the memory math is brutal even for a model as small as Glimmer. Full fine-tuning needs the weights (~60 GB at 16-bit), plus gradients of equal size (~60 GB), plus optimizer state — the Adam optimizer keeps two auxiliary numbers per parameter, ~240 GB — plus activations. Several hundred gigabytes for a 30-billion-

> parameter model, on a good day. This is why Section 6 exists: almost nobody training a private model does full fine-tuning, and the alternatives were designed precisely around this wall.

5. The Scale Shock: Muse Spark Open

If Glimmer is the model you can own, Muse Spark Open is the model that shows you what "own" costs at the frontier. Spark Open is an 816-billion-parameter mixture-of-experts model (42 billion parameters active per token), released as open weights in October 2026. The numbers:

Precision	Weight footprint	Hardware required
4-bit	~408 GB	Multi-GPU node or 748 GB deskside
8-bit	~816 GB	Does not fit in any single deskside machine
16-bit (full)	~1.6 TB	Datacenter node (e.g. 8×B300, 2.3 TB)

Table 2 — Approximate footprint of Muse Spark Open. Even the 4-bit build exceeds every sub-$10,000 machine.

Concretely: an NVIDIA DGX Spark (128 GB unified, ~$4,700) cannot run it at any precision. The cheapest single box that can is Dell's GB300 deskside at **$175,496.74**. Below that, the route is rented: eight H100s at roughly $16–24 per hour, spun up and shut down — nobody runs the full build continuously except Meta. Note the 8-bit row: 816 billion parameters at one byte each is 816 GB, which exceeds the 748 GB of the most expensive deskside box sold. The 4-bit build is not a choice; it is the *only* build that runs outside a datacenter.

This is the economic consequence nobody planned. Every step of this paper — the bandwidth wall, the precision ladder, the training memory wall — is a statement about *memory*: how much of it, how fast, in one place. Multiply those requirements by every company, lab, and enthusiast who wants private AI, and you get the insatiable demand for high-bandwidth memory that has defined the hardware market of 2026 — the DRAM shortage, the doubled GPU prices, the $175,000 deskside computer. The memory wall is not just a technical constraint on your project. It is the central economic fact of the AI era.

6. Training Strategy: Size Decides

Once the model runs, the question becomes whether to train it — to adapt it to your data, your domain, your way of working. The field's standard guidance is correct but incomplete: *fine-tune for behaviour, retrieve for knowledge*. Retrieval-augmented generation (RAG) — fetching relevant documents into the model's context at query time [2] — supplies facts without touching weights: always current, always citable, no retraining when documents change. Fine-tuning teaches *how*: reasoning patterns, tone, task formats. What the guidance omits is the variable that decides how it applies: **the size of the base model**.

> **Small model: train, and train freely.** A weak base model has headroom — large relative gains from training because there is much missing capability to add. Full fine-tuning is

worthwhile; combine it with RAG (train the behaviour, retrieve the facts); iterate across multiple rounds, merging adapters into the base between rounds. The smaller the model, the better the idea full-weights training is.

Large model: retrieve first. A strong base model has little to gain and much to lose. RAG alone covers most needs — most practitioners who believe they need fine-tuning actually need retrieval. Reserve low-rank adaptation (LoRA) [3] for *behaviour* change, keep passes few, and validate general capability between rounds, because each round measurably erodes it.

Boundary condition: "small" and "large" are relative to the task. Glimmer is large relative to document Q&A and small relative to frontier reasoning. Judge by the headroom the task leaves, not the parameter count on the box.

Three mechanisms justify the asymmetry. **First, catastrophic forgetting, and the proportionality of its tax** [4]: every training round on narrow data pulls weights toward the specialty and away from everything else. The tax is levied against the account it is drawn on — small for a small model, ruinous against the millions of dollars of capability baked into a large one. **Second, training on unfamiliar facts teaches guessing, not knowing**: Gekhman et al. showed that fine-tuning on knowledge absent from pre-training produces fluent, confident confabulation that generalizes beyond the training data [5] — the empirical backbone of "retrieve, don't bake." **Third, in-context ability closes the gap**: large models read retrieved context with a fidelity that makes weight-level knowledge unnecessary, so the training-free alternative improves with size even as training's risks grow.

The Glimmer-vs-Spark-Open contrast makes the strategy concrete. Deploying **Glimmer** privately: RAG over your documents on the base model, LoRA only if its behaviour disappoints, a handful of rounds at most. Deploying **Spark Open**: RAG is essentially the *only* move — full training is a datacenter operation, LoRA at this scale is a research project, and the base model is already stronger than anything narrow training would produce. Same rule, both ends of it, one mechanism.

7. Practical Guidance

- **Start with RAG.** Index your documents, retrieve at query time, evaluate. Add training only if *behaviour* — not knowledge — disappoints.
- **Prefer adapters.** LoRA trains small adapter matrices while the base weights stay frozen; iterated merge-and-retrain (ReLoRA [6]) compounds gains without compounding risk the way repeated full fine-tuning does.
- **Replay.** Mix general-purpose data (even 5–10%) into every training round. You are setting the exchange rate between specialty and generality — set it deliberately.
- **Validate generality between rounds.** Track broad capability alongside task scores. If general scores fall while task scores rise, you are spending the account, not growing it.

- **Budget the dataset, not the GPU.** Good training needs thousands of quality prompt→response pairs demonstrating the behaviour you want. Curating them is the real labour; the GPU time is the cheap part.

8. Conclusion

Private AI — a model you own, on hardware you control, trained or retrieved on data no one else sees — is genuinely achievable in 2026, and this paper has tried to show the whole price tag in one place. The mathematics demands memory: full weight matrices streaming per token, bandwidth as the binding constraint, no distributed shortcut as there was for hashing. The precision ladder converts that mathematics into purchase decisions. The training strategy converts the running model into *your* model — with the size of the base model deciding how far training should go. None of this is conceptually difficult. It is just rarely written down together, which is why every newcomer rediscovers it the expensive way. Now it is written down.

References

[1] Vaswani, A., et al. "Attention Is All You Need." *Advances in Neural Information Processing Systems* 30, 2017.

[2] Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems* 33, 2020.

[3] Hu, E. J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." *International Conference on Learning Representations*, 2022 (arXiv 2021).

[4] Kirkpatrick, J., et al. "Overcoming Catastrophic Forgetting in Neural Networks." *Proceedings of the National Academy of Sciences* 114(13), 2017.

[5] Gekhman, Z., et al. "Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations?" *arXiv: 2405.05904*, 2024.

[6] Lialin, V., et al. "ReLoRA: High-Rank Training Through Low-Rank Updates." *arXiv:2308.11411*, 2023.